

使用 IPv6 靜態路徑下一個躍點執行個體 (未標記和已標記)、下一個躍點位址和下一個躍點閘道

來源：<https://codelabs.developers.google.com/ipv6-staticrouting-nexthopilb?hl=zh-tw>

步驟數：10

在這個程式碼研究室中，您將瞭解如何使用 IPv6 靜態路徑搭配新的下一個躍點屬性，例如 `next-hop-instance`、`next-hop-gateway` 和 `next-hop-address`

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 設定用戶端虛擬私有雲](#)
4. [4. 設定伺服器虛擬私有雲](#)
5. [5. 建立閘道執行個體](#)
6. [6. 建立負載平衡器元件](#)
7. [7. 建立及測試負載平衡器的路徑 \(使用負載平衡器位址\)](#)
8. [8. 建立及測試負載平衡器的路徑 \(使用負載平衡器名稱\)](#)
9. [9. 清理](#)
10. [10. 恭喜](#)

步驟 1 · 約 0 分鐘

使用 IPv6 靜態路徑的下一個躍點執行個體 (無標記和已標記)、下一個躍點位址和下一個躍點閘道

程式碼研究室簡介

*subject*上次更新時間：3月 27, 2026

*account_circle*作者：Ghaleb Al-Habian

1. 簡介

靜態自訂路徑會影響虛擬私有雲中的預設轉送行為。IPv6 自訂路徑現在支援新的下一個躍點屬性：next-hop-gateway、next-hop-instance 和 next-hop-address。本程式碼實驗室說明如何使用 IPv6 自訂路徑，搭配這些新的下一個躍點選項，並透過多個 NIC VM 執行個體連線兩個 VPC。您也會示範如何混合使用 ULA 和 GUA 位址，以及如何使用新的自訂路由功能，為 ULA 虛擬私有雲提供公開網際網路的可連性。

課程內容

- 如何建立 IPv6 自訂路由，並指定 ILB 名稱，將下一個躍點設為 next-hop-ilb
- 如何建立 IPv6 自訂路由，並指定 ILB 的 IPv6 位址做為下一個躍點

軟硬體需求

- Google Cloud 專案
-

步驟 2 · 約 0 分鐘

2. 事前準備

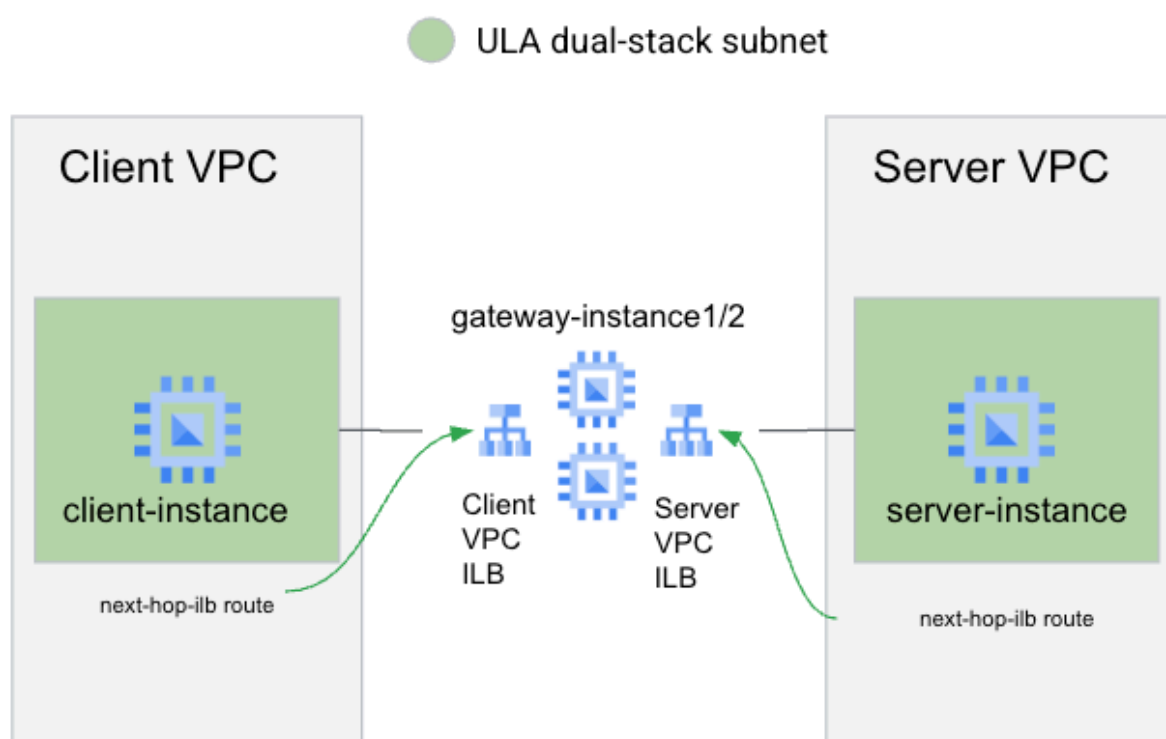
更新專案以支援 Codelab

本程式碼研究室會使用 `$variables`，協助您在 Cloud Shell 中實作 `gcloud` 設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
export projectname=$(gcloud config list --format="value(core.project)")
```

實驗室整體架構



為示範這兩種自訂路由的下一個躍點，您將建立 2 個虛擬私有雲：使用 ULA 位址的用戶端和伺服器虛擬私有雲。

如要讓用戶端虛擬私有雲存取伺服器，您將使用自訂路由，並透過 `next-hop-ilb` 指向一組多重 NIC 閘道執行個體前端的 ILB (使用 ILB 的名稱)，這些執行個體夾在兩個 ILB 之間。如要提供路由，將流量導回用戶端執行個體 (刪除預設的 `::/0` 路由後)，您將使用自訂路由，並透過 `next-hop-ilb` (使用 ILB 的位址) 指向 ILB。

步驟 3 · 約 10 分鐘

3. 設定用戶端虛擬私有雲

建立用戶端虛擬私有雲

在 Cloud Shell 中執行下列操作：

□□□

```
gcloud compute networks create client-vpc \  
  --project=$projectname \  
  --subnet-mode=custom --mtu=1500 \  
  --bgp-routing-mode=regional \  
  --enable-ula-internal-ipv6
```

建立 Client 子網路

在 Cloud Shell 中執行下列操作：

□□□

```
gcloud compute networks subnets create client-subnet \  
  --network=client-vpc \  
  --project=$projectname \  
  --range=192.168.1.0/24 \  
  --stack-type=IPV4_IPV6 \  
  --ipv6-access-type=internal \  
  --region=us-central1
```

使用下列指令，在環境變數中記錄指派的 IPv6 子網路

□□□

```
export client_subnet=$(gcloud compute networks subnets \  
  describe client-subnet \  
  --project $projectname \  
  --format="value(internalIpv6Prefix)" \  
  --region us-central1)
```

啟動用戶端執行個體

在 Cloud Shell 中執行下列操作：

□□□

```
gcloud compute instances create client-instance \  
  --subnet client-subnet \  
  --stack-type IPV4_IPV6 \  
  --zone us-central1-a \  
  --project=$projectname
```

為用戶端 VPC 流量新增防火牆規則

在 Cloud Shell 中執行下列操作：

```
gcloud compute firewall-rules create allow-gateway-client \
  --direction=INGRESS --priority=1000 \
  --network=client-vpc --action=ALLOW \
  --rules=tcp --source-ranges=$client_subnet \
  --project=$projectname
```

新增防火牆規則，允許用戶端執行個體使用 IAP

在 Cloud Shell 中執行下列操作：

```
gcloud compute firewall-rules create allow-iap-client \
  --direction=INGRESS --priority=1000 \
  --network=client-vpc --action=ALLOW \
  --rules=tcp:22 --source-ranges=35.235.240.0/20 \
  --project=$projectname
```

確認用戶端執行個體的 SSH 存取權

在 Cloud Shell 中，登入用戶端執行個體：

```
gcloud compute ssh client-instance \
  --project=$projectname \
  --zone=us-central1-a \
  --tunnel-through-iap
```

如果成功，您會看到用戶端執行個體的終端機視窗。退出 SSH 工作階段，繼續進程式碼研究室。

步驟 4 · 約 10 分鐘

4. 設定伺服器虛擬私有雲

建立伺服器 VPC

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create server-vpc \  
  --project=$projectname \  
  --subnet-mode=custom --mtu=1500 \  
  --bgp-routing-mode=regional \  
  --enable-ula-internal-ipv6
```

建立伺服器子網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create server-subnet \  
  --network=server-vpc \  
  --project=$projectname \  
  --range=192.168.0.0/24 \  
  --stack-type=IPV4_IPV6 \  
  --ipv6-access-type=internal \  
  --region=us-central1
```

使用下列指令，將指派的子網路記錄在環境變數中

```
export server_subnet=$(gcloud compute networks subnets \  
  describe server-subnet \  
  --project $projectname \  
  --format="value(internalIpv6Prefix)" \  
  --region us-central1)
```

啟動伺服器 VM

在 Cloud Shell 中執行下列操作：

```
gcloud compute instances create server-instance \  
  --subnet server-subnet \  
  --stack-type IPV4_IPV6 \  
  --zone us-central1-a \  
  --project=$projectname
```

新增防火牆規則，允許用戶端存取伺服器

在 Cloud Shell 中執行下列操作：

```
gcloud compute firewall-rules create allow-client-server \
  --direction=INGRESS --priority=1000 \
  --network=server-vpc --action=ALLOW \
  --rules=tcp --source-ranges=$client_subnet \
  --project=$projectname
```

新增防火牆規則，允許 IAP

在 Cloud Shell 中執行下列操作：

```
gcloud compute firewall-rules create allow-iap-server \
  --direction=INGRESS --priority=1000 \
  --network=server-vpc --action=ALLOW \
  --rules=tcp:22 \
  --source-ranges=35.235.240.0/20 \
  --project=$projectname
```

在 ULA 伺服器執行個體中安裝 Apache

在 Cloud Shell 中，登入用戶端執行個體：

```
gcloud compute ssh server-instance \
  --project=$projectname \
  --zone=us-central1-a \
  --tunnel-through-iap
```

在伺服器 VM 殼層中執行下列指令

```
sudo apt update && sudo apt -y install apache2
```

確認 Apache 是否正在執行

```
sudo systemctl status apache2
```

覆寫預設網頁

```
echo '<!doctype html><html><body><h1>Hello World! From Server Instance!</h1></body></html>' |
sudo tee /var/www/html/index.html
```

結束 SSH 工作階段，繼續進程式碼研究室。

5. 建立閘道執行個體

建立多重 NIC 閘道執行個體範本

在 Cloud Shell 中執行下列操作：

```
gcloud compute instance-templates create gateway-instance-template \
  --project=$projectname \
  --instance-template-region=us-central1 \
  --region=us-central1 \
  --network-interface=stack-type=IPv4_IPv6,subnet=client-subnet,no-address \
  --network-interface=stack-type=IPv4_IPv6,subnet=server-subnet,no-address \
  --can-ip-forward \
  --metadata=startup-script='#!/bin/bash
sudo sysctl -w net.ipv6.conf.ens4.accept_ra=2
sudo sysctl -w net.ipv6.conf.ens5.accept_ra=2
sudo sysctl -w net.ipv6.conf.ens4.accept_ra_defrtr=1
sudo sysctl -w net.ipv6.conf.all.forwarding=1'
```

建立多 NIC 閘道執行個體群組

在 Cloud Shell 中執行下列操作：

```
gcloud compute instance-groups managed create gateway-instance-group \
  --project=$projectname \
  --base-instance-name=gateway-instance \
  --template=projects/$projectname/regions/us-central1/instanceTemplates/gateway-
instance-template \
  --size=2 \
  --zone=us-central1-a
```

驗證閘道執行個體

確保開機指令碼已正確傳遞，且 v6 路由表正確無誤。透過 SSH 連線至其中一個閘道執行個體

在 Cloud Shell 中執行下列指令，列出閘道執行個體：

```
gcloud compute instances list \
  --project=$projectname \
  --zones=us-central1-a \
  --filter name~gateway \
  --format 'csv(name)'
```

記下其中一個執行個體名稱，並在下一個指令中使用，透過 SSH 連線至該執行個體。

在 Cloud Shell 中，登入其中一個閘道執行個體

```
gcloud compute ssh gateway-instance-<suffix> \
  --project=$projectname \
  --zone=us-central1-a \
  --tunnel-through-iap
```

在閘道 VM 殼層中，執行下列指令來檢查 IPv6 轉送

```
sudo sysctl net.ipv6.conf.all.forwarding
```

指令應傳回值「1」，表示已啟用 IPv6 轉送。

驗證執行個體上的 IPv6 路由表

```
ip -6 route show
```

範例輸出內容，顯示 ULA 和 GUA 子網路路徑，預設路徑指向 GUA 介面。

```
:::1 dev lo proto kernel metric 256 pref medium
2600:1900:4000:7a7f:0:1:: dev ens4 proto kernel metric 256 expires 83903sec pref medium
2600:1900:4000:7a7f::/65 via fe80::4001:c0ff:fea8:101 dev ens4 proto ra metric 1024 expires
88sec pref medium
fd20:3df:8d5c::1:0:0 dev ens5 proto kernel metric 256 expires 83904sec pref medium
fd20:3df:8d5c::/64 via fe80::4001:c0ff:fea8:1 dev ens5 proto ra metric 1024 expires 84sec
pref medium
fe80::/64 dev ens5 proto kernel metric 256 pref medium
fe80::/64 dev ens4 proto kernel metric 256 pref medium
default via fe80::4001:c0ff:fea8:101 dev ens4 proto ra metric 1024 expires 88sec pref medium
```

退出 SSH 工作階段，繼續進程式碼研究室。

6. 建立負載平衡器元件

我們必須先在閘道執行個體兩端建立內部直通負載平衡器來轉送流量，才能在兩個虛擬私有雲中建立路由。

本程式碼研究室建立的負載平衡器包含

- 健康狀態檢查：在本程式碼研究室中，我們將建立以通訊埠 22 為目標的簡單健康狀態檢查。請注意，健康狀態檢查不會在部署後運作 (這會涉及新增防火牆規則，允許健康狀態檢查，並在閘道執行個體上建立特殊路由)。由於本程式碼研究室的重點是 IPv6 轉送，因此當所有後端都不健康時，我們會依賴[內部直通負載平衡器的預設流量分配行為](#)，也就是在不得已時轉送至所有後端。
- 後端服務：我們將為後端服務使用 TCP 通訊協定。但由於負載平衡器是為路由用途而建立，因此無論後端服務通訊協定為何，所有通訊協定都會轉送。
- 轉送規則：我們會為每個虛擬私有雲建立轉送規則。
- 內部 IPv6 位址：在本程式碼研究室中，我們將讓轉送規則從子網路自動分配 IPv6 位址

建立健康狀態檢查

在 Cloud Shell 中執行下列操作：

```
gcloud compute health-checks create tcp tcp-hc-22 \  
  --project=$projectname \  
  --region=us-central1 \  
  --port=22
```

建立後端服務

在 Cloud Shell 中執行下列操作：

```
gcloud compute backend-services create bes-ilb-clientvpc \  
  --project=$projectname \  
  --load-balancing-scheme=internal \  
  --protocol=tcp \  
  --network=client-vpc \  
  --region=us-central1 \  
  --health-checks=tcp-hc-22 \  
  --health-checks-region=us-central1  
  
gcloud compute backend-services create bes-ilb-servervpc \  
  --project=$projectname \  
  --load-balancing-scheme=internal \  
  --protocol=tcp \  
  --network=server-vpc \  
  --region=us-central1 \  
  --health-checks=tcp-hc-22 \  
  --health-checks-region=us-central1
```

將執行個體群組新增至後端服務

在 Cloud Shell 中執行下列操作：

```
gcloud compute backend-services add-backend bes-ilb-clientvpc \  
  --project=$projectname \  
  --region=us-central1 \  
  --instance-group=gateway-instance-group \  
  --instance-group-zone=us-central1-a  
gcloud compute backend-services add-backend bes-ilb-servervpc \  
  --project=$projectname \  
  --region=us-central1 \  
  --instance-group=gateway-instance-group \  
  --instance-group-zone=us-central1-a
```

建立轉送規則

在 Cloud Shell 中執行下列操作：

```
gcloud compute forwarding-rules create fr-ilb-clientvpc \  
  --project=$projectname \  
  --region=us-central1 \  
  --load-balancing-scheme=internal \  
  --network=client-vpc \  
  --subnet=client-subnet \  
  --ip-protocol=TCP \  
  --ip-version=IPV6 \  
  --ports=ALL \  
  --backend-service=bes-ilb-clientvpc \  
  --backend-service-region=us-central1  
  
gcloud compute forwarding-rules create fr-ilb-servervpc \  
  --project=$projectname \  
  --region=us-central1 \  
  --load-balancing-scheme=internal \  
  --network=server-vpc \  
  --subnet=server-subnet \  
  --ip-protocol=TCP \  
  --ip-version=IPV6 \  
  --ports=ALL \  
  --backend-service=bes-ilb-servervpc \  
  --backend-service-region=us-central1
```

在 Cloud Shell 中發出下列指令，記錄這兩項轉送規則的 IPv6 位址：

```
export fraddress_client=$(gcloud compute forwarding-rules \  
  describe fr-ilb-clientvpc \  
  --project $projectname \  
  --format="value(IPAddress)" \  
  --region us-central1)  
  
export fraddress_server=$(gcloud compute forwarding-rules \  
  describe fr-ilb-servervpc \  
  --project $projectname \  
  --format="value(IPAddress)" \  
  --region us-central1)
```

步驟 7 · 約 10 分鐘

7. 建立及測試負載平衡器的路徑 (使用負載平衡器位址)

在本節中，您將使用負載平衡器的 IPv6 位址做為下一個躍點，在用戶端和伺服器虛擬私有雲中新增路徑。

記下伺服器位址

在 Cloud Shell 中執行下列操作：

```
gcloud compute instances list \
  --project $projectname \
  --zones us-central1-a \
  --filter="name~server-instance" \
  --format='value[separator=", "](name,networkInterfaces[0].ipv6Address)'
```

這應該會輸出伺服器執行個體名稱和 IPv6 前置字元。輸出範例

```
server-instance,fd20:3df:8d5c:0:0:0:0:0
```

請記下伺服器位址，稍後會從用戶端執行個體使用 curl 指令。很抱歉，環境變數無法輕鬆用於儲存這些項目，因為環境變數不會透過 SSH 工作階段傳輸。

從用戶端對 ULA 伺服器執行個體執行 curl 指令

如要查看新增任何路徑前的行為，從用戶端執行個體對 server-instance1 執行 curl 指令。

在 Cloud Shell 中，登入用戶端執行個體：

```
gcloud compute ssh client-instance \
  --project=$projectname \
  --zone=us-central1-a \
  --tunnel-through-iap
```

在用戶端執行個體中，使用 server1 執行個體的 ULA IPV6 位址執行 curl (指令會將逾時時間設為 5 秒，避免 curl 等待時間過長)

```
curl -m 5.0 -g -6 'http://[ULA-ipv6-address-of-server1]:80/'
```

附註：請務必使用 curl 指令的 IPv6 對等項目：

```
curl -g -6 'http://[ip-address-of-server]:80/'
```

範例：`curl -g -6 'http://[fd20:987:cc5e::]:80/**'`

由於用戶端 VPC 尚未有通往伺服器 VPC 的路徑，因此這個 curl 指令應會逾時。

讓我們試著解決這個問題！暫時結束 SSH 工作階段。

在用戶端 VPC 中新增自訂路由

由於用戶端 VPC 缺少前往 ULA 前置字串的路徑，現在，請建立指向用戶端 ILB 的路由 (依位址)，加入該路由。

注意：系統會為 IPv6 內部直通負載平衡器指派 /96 位址。您必須先從地址中移除 /96 遮罩，才能將地址傳遞至下一個指令。(以下使用 bash 內建替代)

在 Cloud Shell 中執行下列操作：

```
gcloud compute routes create client-to-server-route \
  --project=$projectname \
  --destination-range=$server_subnet \
  --network=client-vpc \
  --next-hop-ilb=${fraddress_client//\//96}
```

使用 SSH 返回用戶端執行個體：

```
gcloud compute ssh client-instance \
  --project=$projectname \
  --zone=us-central1-a \
  --tunnel-through-iap
```

在用戶端執行個體中，再次嘗試對伺服器執行個體執行 curl。(此指令會將逾時時間設為 5 秒，避免 curl 等待過久)

```
curl -m 5.0 -g -6 'http://[ULA-ipv6-address-of-server1]:80/'
```

這個 curl 指令仍會逾時，因為伺服器 VPC 尚未透過閘道執行個體，將路由傳回用戶端 VPC。

退出 SSH 工作階段，繼續進程式碼研究室。

在伺服器虛擬私有雲中新增自訂路由

在 Cloud Shell 中執行下列操作：

```
gcloud compute routes create server-to-client-route \
  --project=$projectname \
  --destination-range=$client_subnet \
  --network=server-vpc \
  --next-hop-ilb=${fraddress_server//\//96}
```

使用 SSH 返回用戶端執行個體：

```
gcloud compute ssh client-instance \
  --project=$projectname \
  --zone=us-central1-a \
  --tunnel-through-iap
```

在用戶端執行個體中，再次嘗試對伺服器執行個體執行 curl。

```
curl -m 5.0 -g -6 'http://[ULA-ipv6-address-of-server1]:80/'
```

這個 curl 指令現在會成功執行，表示您已從用戶端執行個體端對端連線至 ULA 伺服器執行個體。目前只能透過使用 IPv6 自訂路徑，並將 next-hop-ilb 設為下一個躍點，才能建立連線。

輸出內容範例

```
<user id>@client-instance:~$ curl -m 5.0 -g -6 'http://[fd20:3df:8d5c:0:0:0:0]:80/'  
<!doctype html><html><body><h1>Hello World! From Server Instance!</h1></body></html>
```

退出 SSH 工作階段，繼續進程式碼研究室。

步驟 8 · 約 10 分鐘

8. 建立及測試負載平衡器的路徑 (使用負載平衡器名稱)

或者，next-hop-ilb 也可以參照負載平衡器的名稱，而非 IPv6 位址。本節將說明相關程序，並測試用戶端與伺服器之間是否仍能建立連線。

刪除先前的路線

讓我們刪除使用執行個體名稱的自訂路徑，將環境還原至新增任何自訂路徑之前的狀態。

在 Cloud Shell 中執行下列操作：

```
gcloud compute routes delete client-to-server-route --quiet --project=$projectname
gcloud compute routes delete server-to-client-route --quiet --project=$projectname
```

從用戶端對 ULA 伺服器執行個體執行 curl 指令

如要確認先前的路由已成功刪除，請從用戶端執行個體對 server-instance1 執行 curl 指令。

在 Cloud Shell 中，登入用戶端執行個體：

```
gcloud compute ssh client-instance \
  --project=$projectname \
  --zone=us-central1-a \
  --tunnel-through-iap
```

在用戶端執行個體中，使用 server1 執行個體的 ULA IPV6 位址執行 curl (指令會將逾時時間設為 5 秒，避免 curl 等待時間過長)

```
curl -m 5.0 -g -6 'http://[ULA-ipv6-address-of-server1]:80/'
```

附註：請務必使用 curl 指令的 IPv6 對等項目：

```
curl -g -6 'http://[ip-address-of-server]:80/'
```

範例：`curl -g -6 'http://[fd20:987:cc5e::]:80/'**`

由於用戶端 VPC 不再有通往伺服器 VPC 的路徑，因此這個 curl 指令應會逾時。

在用戶端和伺服器虛擬私有雲中新增自訂路徑

讓我們在用戶端和伺服器虛擬私有雲中重新新增自訂路徑，但我們會在指令中使用 ILB 的名稱和區域，而不是 ILB 的位址。

在 Cloud Shell 中執行下列操作：

```
gcloud compute routes create client-to-server-route \
  --project=$projectname \
  --destination-range=$server_subnet \
  --network=client-vpc \
  --next-hop-ilb=fr-ilb-clientvpc \
  --next-hop-ilb-region=us-central1

gcloud compute routes create server-to-client-route \
  --project=$projectname \
  --destination-range=$client_subnet \
  --network=server-vpc \
  --next-hop-ilb=fr-ilb-servervpc \
  --next-hop-ilb-region=us-central1
```

使用 SSH 返回用戶端執行個體：

```
gcloud compute ssh client-instance \
  --project=$projectname \
  --zone=us-central1-a \
  --tunnel-through-iap
```

在用戶端執行個體中，再次嘗試對伺服器執行個體執行 curl。(此指令會將逾時時間設為 5 秒，避免 curl 等待過久)

```
curl -m 5.0 -g -6 'http://[ULA-ipv6-address-of-server1]:80/'
```

這個 curl 指令現在會成功執行，表示您已從用戶端執行個體端對端連線至 ULA 伺服器執行個體。

步驟 9 · 約 0 分鐘

9. 清理

清理自訂路徑

在 Cloud Shell 中執行下列操作：

```
gcloud compute routes delete client-to-server-route --quiet --project=$projectname
gcloud compute routes delete server-to-client-route --quiet --project=$projectname
```

清理 LB 元件

在 Cloud Shell 中執行下列操作：

```
gcloud compute forwarding-rules delete fr-ilb-clientvpc --region us-central1 --quiet --
project=$projectname
gcloud compute forwarding-rules delete fr-ilb-servervpc --region us-central1 --quiet --
project=$projectname

gcloud compute backend-services delete bes-ilb-clientvpc --region us-central1 --quiet --
project=$projectname
gcloud compute backend-services delete bes-ilb-servervpc --region us-central1 --quiet --
project=$projectname

gcloud compute health-checks delete tcp-hc-22 --region us-central1 --quiet --
project=$projectname
```

清理執行個體和執行個體範本

在 Cloud Shell 中執行下列操作：

```
gcloud compute instances delete client-instance --zone us-central1-a --quiet --
project=$projectname

gcloud compute instances delete server-instance --zone us-central1-a --quiet --
project=$projectname

gcloud compute instance-groups managed delete gateway-instance-group --zone us-central1-a --
quiet --project=$projectname

gcloud compute instance-templates delete gateway-instance-template --region us-central1 --
quiet --project=$projectname
```

清理子網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets delete client-subnet --region=us-central1 --quiet --project=$projectname
```

```
gcloud compute networks subnets delete server-subnet --region=us-central1 --quiet --project=$projectname
```

清理防火牆規則

在 Cloud Shell 中執行下列操作：

```
gcloud compute firewall-rules delete allow-iap-client --quiet --project=$projectname
gcloud compute firewall-rules delete allow-iap-server --quiet --project=$projectname
gcloud compute firewall-rules delete allow-gateway-client --quiet --project=$projectname
gcloud compute firewall-rules delete allow-client-server --quiet --project=$projectname
```

清理虛擬私有雲

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks delete client-vpc --quiet --project=$projectname
gcloud compute networks delete server-vpc --quiet --project=$projectname
```

步驟 10 · 約 0 分鐘

10. 恭喜

您已成功使用靜態自訂 IPv6 路徑，並將下一個躍點設為 next-hop-ilb。您也使用這些路徑驗證了端對端 IPv6 通訊。

後續步驟

查看一些程式碼研究室...

- [使用 IPv6 位址從地端部署主機存取 Google API](#)
- [IP 定址選項：IPv4 和 IPv6](#)
- [使用 IPv6 靜態路由的下一個躍點執行個體、下一個躍點位址和下一個躍點閘道](#)

延伸閱讀與影片

- [Google Cloud 上的 IPv6 簡介](#)

參考文件

- [支援靜態路徑的 IPv6](#)
 - [IPv6 子網路](#)
-